LEARN BEING FORWARD

Programming — Java Programming

# Collections framework

COMPLETE LEARNING GUIDE

### What You'll Learn

Almost no real Java program manages data with plain arrays — the Collections Framework's Lists, Sets, Maps, and Queues are the actual day-to-day data structures used for everything from caching to request handling, and knowing which implementation to reach for (and why) is a constant, practical decision rather than academic trivia. This sub-module walks through the core interface hierarchy, compares the standard List, Set, and Map implementations by their underlying data structure and performance characteristics, covers safe iteration (including the fail-fast ConcurrentModificationException every Java developer eventually hits), sorting with Comparable and Comparator, and closes with generics and a first taste of the Streams API, since modern Java code almost never processes a collection with a raw for-loop anymore.

## 1. The Collection Hierarchy: List, Set, Map, and Queue

The java.util.Collection interface is the root of List, Set, and Queue — each representing a group of elements, but with different rules: List preserves insertion order and allows duplicates, Set enforces uniqueness, and Queue models a processing order (typically FIFO). Map is deliberately not a Collection at all; it's a separate hierarchy of key-value pairs, since a Map's fundamental operation (lookup by key) doesn't fit the 'collection of elements' abstraction the others share. The utility class Collections (plural, with a capital C and a trailing 's') provides static helper methods like sort, reverse, and unmodifiableList — a frequent source of confusion is mixing up this utility class with the Collection interface it operates on.

- Collection (interface, singular) vs Collections (utility class, plural, static helper methods) — easy to confuse by name alone.
- Map is not a Collection — it has its own separate interface hierarchy because key-based lookup doesn't fit the Collection contract.

## 2. ArrayList vs LinkedList

ArrayList is backed by a resizable internal array, giving O(1) index-based access (get(i)) but O(n) insertion or removal in the middle, since every following element must shift. LinkedList is a doubly-linked list of nodes, giving O(1) insertion/removal at either end but O(n) access by index, since reaching element i requires walking the chain from one end. In practice, ArrayList is the default choice for nearly all use cases because random access and iteration dominate real workloads, while LinkedList is reserved for scenarios with genuinely frequent insertions/removals at the ends, such as implementing a Deque-style structure.

```java
List<String> names = new ArrayList<>();
names.add("Asha");
names.add("Raj");
names.add(0, "Priya"); // O(n) — shifts "Asha" and "Raj" right

List<String> queue = new LinkedList<>();
queue.add("first");
((LinkedList<String>) queue).addFirst("even-earlier"); // O(1) on LinkedList
```

## 3. Set Implementations: HashSet, LinkedHashSet, TreeSet

HashSet stores elements in a hash table, giving O(1) average-case add/contains/remove but no guaranteed iteration order. LinkedHashSet adds a linked list alongside the hash table purely to preserve insertion order during iteration, at a small memory cost. TreeSet stores elements in a red-black tree, keeping them continuously sorted (via Comparable or a supplied Comparator) at O(log n) per operation, and additionally implements NavigableSet methods like higher(), lower(), and first() for range-style queries. Choosing between them is purely about what ordering guarantee (if any) the problem actually needs.

```java
Set<String> unordered = new HashSet<>(List.of("c", "a", "b"));
Set<String> insertionOrder = new LinkedHashSet<>(List.of("c", "a", "b"));
Set<String> sorted = new TreeSet<>(List.of("c", "a", "b"));

System.out.println(insertionOrder); // [c, a, b]
System.out.println(sorted);         // [a, b, c]
```

## 4. Map Implementations: HashMap, LinkedHashMap, TreeMap

The Map family mirrors the Set family's tradeoffs, since Java's HashSet is actually implemented internally using a HashMap with dummy values. HashMap allows exactly one null key and any number of null values, with O(1) average get/put but unspecified iteration order; LinkedHashMap preserves insertion order; TreeMap keeps keys continuously sorted. Beyond basic get/put, modern Map methods like computeIfAbsent, merge, and getOrDefault eliminate a lot of the manual 'check if key exists, then put' boilerplate that used to be ubiquitous in older Java code.

```java
Map<String, Integer> wordCounts = new HashMap<>();
String[] words = {"cat", "dog", "cat", "bird", "dog", "cat"};

for (String w : words) {
    wordCounts.merge(w, 1, Integer::sum); // adds 1, or inserts 1 if absent
}
System.out.println(wordCounts); // {bird=1, cat=3, dog=2} (order not guaranteed)
```

## 5. Iterating Safely: Iterator and ConcurrentModificationException

Standard collection iterators are fail-fast: if a collection is structurally modified (an element added or removed) while a for-each loop or explicit Iterator is mid-traversal — by any means other than the iterator's own remove() — the next call to next() throws a ConcurrentModificationException rather than silently producing wrong results. The correct way to remove elements while iterating is either Iterator.remove() directly, or the simpler Collection.removeIf(predicate) introduced in Java 8, both of which update the collection's internal state consistently.

```java
List<Integer> nums = new ArrayList<>(List.of(1, 2, 3, 4, 5, 6));

// WRONG — throws ConcurrentModificationException
// for (Integer n : nums) { if (n % 2 == 0) nums.remove(n); }

// CORRECT — removeIf handles the bookkeeping safely
nums.removeIf(n -> n % 2 == 0);
System.out.println(nums); // [1, 3, 5]
```

## 6. Sorting with Comparable and Comparator

A class implements Comparable<T> and its compareTo(T other) method to define a single natural ordering intrinsic to the class itself (e.g., sorting Employee by ID). A Comparator<T> is a separate object defining an ordering externally — useful when you need multiple different orderings of the same type, or want to sort a class you don't own. The Comparator interface's static and default methods (comparing, thenComparing, reversed) let you build multi-key sort logic declaratively instead of writing a long if/else compareTo body by hand.

```java
record Employee(String name, int age, double salary) {}

List<Employee> employees = new ArrayList<>(List.of(
    new Employee("Asha", 29, 75000),
    new Employee("Raj", 29, 68000),
    new Employee("Priya", 34, 82000)
));

employees.sort(
    Comparator.comparingInt(Employee::age)
        .thenComparing(Comparator.comparingDouble(Employee::salary).reversed())
);
```

## 7. Generics and Type Safety

Generics let a collection declare the type of element it holds at compile time — List<String> can only ever hold Strings, and the compiler rejects any attempt to add something else, catching a whole category of bugs that used to surface only as a ClassCastException at runtime in pre-generics Java. The diamond operator (<>) lets the compiler infer the type argument on the right side from context, avoiding redundant repetition. Wildcard bounds (? extends Number for read-only producer parameters, ? super Integer for write-only consumer

parameters) show up in method signatures that need to accept a range of related generic types flexibly, though for most day-to-day application code, simple concrete type parameters cover the vast majority of cases.

```java
List<String> names = new ArrayList<>(); // diamond infers <String> from the left side
names.add("Asha");
// names.add(42); // compile error — type safety enforced before runtime
```

## 8. A First Look at the Streams API

Streams (java.util.stream) let you express a pipeline of operations over a collection's elements — filter, map, sorted, collect — declaratively, without writing manual loops and temporary accumulator variables. A stream itself holds no data and can only be consumed once; it describes a computation that executes when a terminal operation like collect() or forEach() is called. Streams complement the Collections Framework rather than replacing it: you typically start from a List or Set, process it as a stream, and collect the result back into a new collection.

```java
List<String> names = List.of("Asha", "Raj", "Priya", "Bob");

List<String> longUppercaseNames = names.stream()
    .filter(n -> n.length() > 3)
    .map(String::toUpperCase)
    .sorted()
    .collect(java.util.stream.Collectors.toList());

System.out.println(longUppercaseNames); // [ASHA, PRIYA]
```

### Common Pitfalls

- Using ArrayList when the workload needs frequent insertion/removal at the front or middle, repeatedly paying the O(n) shift cost.
- Modifying a List, Set, or Map with the collection's own add/remove methods while iterating it with a for-each loop, triggering ConcurrentModificationException instead of using removeIf or Iterator.remove().
- Overriding equals() on a class without also overriding hashCode(), causing duplicate 'equal' entries in a HashSet or failed lookups in a HashMap.
- Relying on HashMap's iteration order being consistent or meaningful — it's unspecified and can change between JDK versions; use LinkedHashMap when order matters.
- Confusing the Collection interface with the Collections utility class by name alone.
- Writing a custom compareTo() or Comparator that isn't consistent with equals(), causing a TreeSet/TreeMap to silently treat distinct-by-equals elements as duplicates.

- Forgetting that a Stream can only be consumed once — reusing the same stream variable for a second terminal operation throws IllegalStateException.

## Key Takeaways

- List preserves order and duplicates, Set enforces uniqueness, Map stores key-value pairs — pick based on which contract the problem actually needs.
- ArrayList: fast random access, slow middle insert/remove. LinkedList: fast insert/remove at the ends, slow random access.
- Always override equals() and hashCode() together for any class used in a HashSet or as a HashMap key.
- Use Iterator.remove() or Collection.removeIf() to remove elements during iteration — never the collection's own remove() inside a for-each loop.
- Comparable defines one natural ordering on the class itself; Comparator defines external, swappable orderings — chain with thenComparing for multi-key sorts.
- Generics catch type mismatches at compile time instead of surfacing as a ClassCastException at runtime.

## Learn More

**Oracle — The Java Tutorials: Collections**
https://docs.oracle.com/javase/tutorial/collections/index.html

**Oracle Java 17 API — Collection Interface**
https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Collection.html

**Baeldung — Guide to Java 8's Collectors**
https://www.baeldung.com/java-8-collectors